

05 — Mejoras post-EP3 e innovación (Examen)

Proyecto: Plataforma Libro de Clases Digital
Asignatura: DSY1106 — Desarrollo Fullstack III
Evaluación: Examen — mejora del encargo EP3 + innovación
Equipo: Cristian Monsalve / Héctor Olivares
Fecha: 2026-07-13
Base documental: Informe Evaluación Parcial N°3 (01 – 04 de esta carpeta)

1. Contexto del examen

Tras la **presentación N°3 (EP3)**, el equipo incorporó el **feedback del docente** y una **mejora innovadora** sugerida: que al registrar asistencia o publicar una evaluación, el sistema **avise de inmediato a los apoderados** por la mensajería interna del colegio.

Este documento **complementa** el informe EP3 (arquitectura, persistencia y pruebas). No lo reemplaza: actualiza el estado del software y detalla las nuevas capacidades.

Recomendación docente (defensa)

Traer el software **corriendo en un PC propio** o agilizar la presentación del **software, código, BD y pruebas**.

Ver guía operativa: 06_guia_defensa_examen.md .

2. Feedback EP3 → acciones realizadas

Feedback / recomendación	Respuesta del equipo	Evidencia en el software
Agilizar demo: app, código, BD y tests visibles	Checklist de arranque, usuarios demo y ruta de demo grabada	06_guia_defensa_examen.md , README.md raíz
Mejorar en base a presentación N°3 (claridad de roles y flujos reales)	Separación Dirección vs Oficina ; panel docente por curso; confirmaciones en acciones sensibles	RBAC frontend + banners de ayuda

Feedback / recomendación	Respuesta del equipo	Evidencia en el software
Incorporar mejora innovadora (alertas a apoderados)	Avisos automáticos al guardar lista y al crear evaluación	Mensajería TG + endpoints de notify
Fortalecer valor pedagógico / comunicación familia–colegio	Módulo de mensajes + alertas contextuales (alumno, asignatura, fecha, estado)	Conversaciones TG docente–apoderado

3. Mejora innovadora: alertas inmediatas a apoderados

3.1 Problema que resuelve

En un libro de clases digital, el apoderado suele enterarse de ausencias o fechas de prueba **días después** (cuaderno, circular o reunión). La innovación acerca el sistema a la vida real del colegio: **cuando el docente actúa, la familia recibe el aviso al instante** dentro de la misma plataforma (sin depender de SMS/email externos en esta etapa).

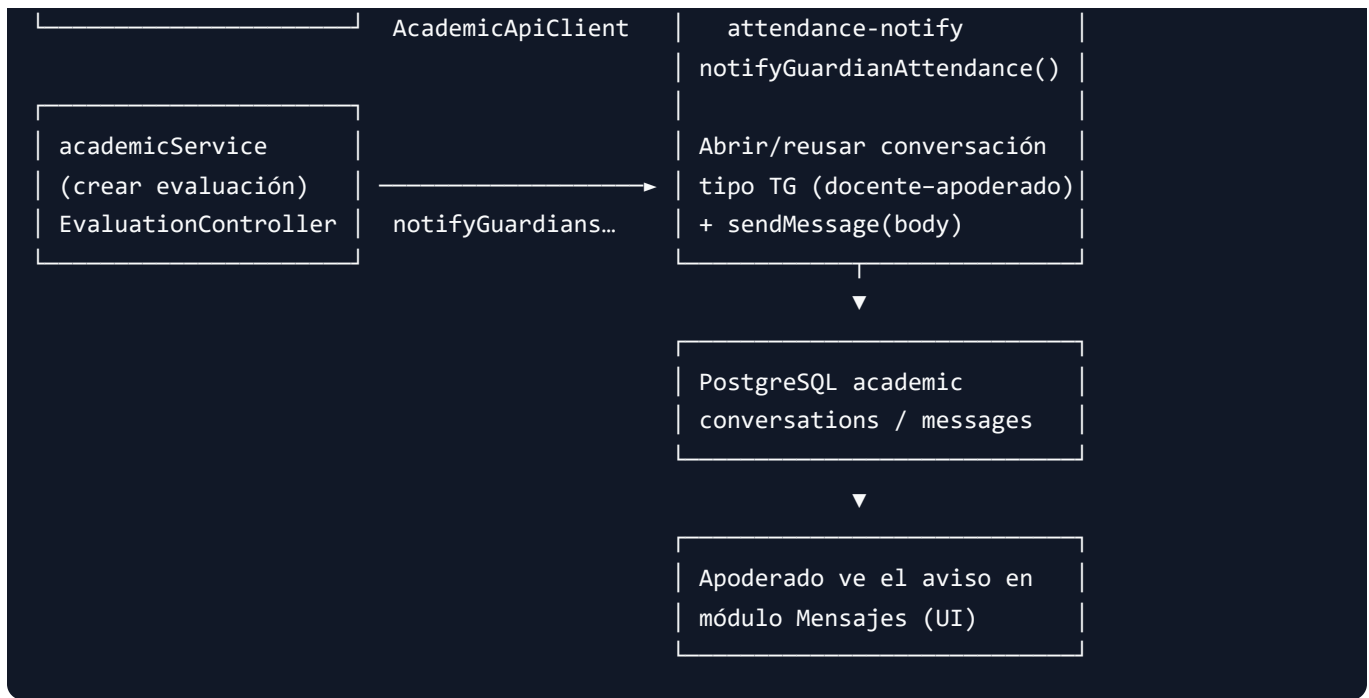
3.2 Idea central

Evento docente	Efecto automático
Guardar registro de asistencia (P / A / R / J)	Mensaje TG al apoderado de cada alumno con apoderado asignado
Crear evaluación (prueba, control, trabajo, examen)	Mensaje TG a apoderados de los alumnos del curso de la asignatura

Ambos flujos **no bloquean** el guardado: si falla el aviso, la asistencia/evaluación se persiste igual (**best-effort**). Antes de confirmar en UI, el docente ve un **diálogo de confirmación** que advierte del aviso a apoderados.

3.3 Arquitectura del flujo (innovación)





3.4 Contenido del aviso (ejemplo)

Asistencia

Aviso de asistencia
Alumno: Juan Perez
Asignatura: Matemática
Fecha de la clase: 13-07-2026
Estado: Ausente (A)

Este mensaje se generó automáticamente al registrar la asistencia de la clase.

Evaluación

Aviso de evaluación
Alumno: Juan Perez
Curso: 1º Medio A
Asignatura: Matemática
Evaluación: Prueba unidad 2
Tipo: PRUEBA
Fecha: 15-07-2026

Este mensaje se generó automáticamente al publicar una nueva evaluación.

3.5 Por qué es innovadora (defensa oral)

1. **Tiempo real pedagógico:** cierra el ciclo docente → familia en segundos.
2. **Misma plataforma:** reutiliza la mensajería interna (conversaciones TG), sin un canal paralelo frágil.
3. **Resiliencia:** el core (asistencia/evaluaciones) no depende del canal de aviso.
4. **Trazabilidad:** el historial queda en la conversación del apoderado (auditoría leve).
5. **UX segura:** `ConfirmDialog` reduce clics accidentales que dispararían notificaciones masivas.
6. **Coherente con microservicios:** attendance no escribe mensajes; **delega** en academic (dueño del dominio de mensajería).

3.6 Archivos / puntos de código clave

Área	Ubicación
Notify asistencia (cliente)	<code>attendanceService/.../client/AcademicApiClient.java</code> → <code>notifyGuardianAttendance</code>
Persistencia + hook lista	Controller/servicio de attendance al crear/actualizar registro
API notify	<code>academicService/.../controller/MessageController.java</code> → POST <code>/messages/attendance-notify</code>
Lógica TG asistencia	<code>MessageServiceImpl.notifyGuardianAttendance</code>
Notify evaluaciones	<code>EvaluationController.createEvaluation</code> → <code>notifyGuardiansEvaluationCreated</code>
UI confirmación	<code>frontend-react/src/components/ConfirmDialog.tsx</code>
Pantallas	<code>Attendance.tsx</code> , <code>Evaluations.tsx</code>

3.6 Demo rápida de la innovación (2–3 min)

1. Login `prof_castillo` / `test1234` → **Asistencia** → pasar lista → confirmar → guardar.
2. Logout → login `apoderado_demo` → **Mensajes** → conversación con el docente → ver aviso.
3. De nuevo docente → **Evaluaciones** → crear prueba → confirmar → crear.
4. Apoderado → Mensajes → ver aviso de evaluación.

4. Otras mejoras incorporadas tras EP3

4.1 Mensajería interna por roles

- Tipos de conversación: oficina–docente, oficina–apoderado, oficina–alumno, **docente–apoderado (TG)**, docente–alumno.
- Distinción visual **Dirección** vs **Oficina** en la UI.

- Base para la innovación (los avisos usan TG).

4.2 Roles afinados (RBAC)

Usuario demo	Rol efectivo	Uso en defensa
admin_colegio	SUPER_ADMINISTRADOR (Dirección)	Estructura, docentes, usuarios, consulta
admin_oficina	ADMINISTRATIVO (Oficina)	Operación / coordinación
prof_castillo / prof_ximena	DOCENTE	Lista, evaluaciones, notas, avisos
apoderado_demo	APODERADO	Mensajes, portal familiar
estudiante_demo	ESTUDIANTE	Portal alumno

Contraseña demo: test1234 .

4.3 Calificaciones y nota final

- Promedio de semestre: CONTROL + PRUEBA + TRABAJO (ponderados).
- Esquema de nota final definido por el colegio (ej. 70 % semestre + 30 % examen).
- Vista tipo planilla + **exportación Excel/PDF**.

4.4 Asistencia reforzada

- Planilla tipo Excel (P/A/R/J), filtros, exportación.
- Admin: **Clases y sustitutos** (crear sesión, filtros curso/asignatura).
- Confirmación al crear registro y al guardar lista.

4.5 Experiencia docente

- Contexto por curso seleccionado (mismo hilo en asistencia, evaluaciones, notas).
- Vinculación ficha docente ↔ usuario auth (email / userId / authUsername) para evitar 403.

4.6 Confirmaciones en acciones sensibles

Componente reutilizable ConfirmDialog en:

- Crear evaluación (con aviso a apoderados)
 - Crear sesión / guardar lista
 - Eliminar evaluación o sesión
-

5. Impacto en arquitectura y persistencia

Aspecto	EP3	Post-EP3 / Examen
Comunicación familia	Consulta de datos en portal	Mensajes + alertas automáticas
academicService	CRUD pedagógico	+ MessageService , notify attendance/evaluation
attendanceService	Sesiones y registros	+ llamada REST a academic al guardar lista
Frontend	CRUD por rol	+ Mensajes, ConfirmDialog, exports, nota final
Persistencia	3 BD	Sin nueva BD: mensajes en academic (conversaciones)

La arquitectura de microservicios del documento 01 se mantiene; la innovación es un **caso de orquestación ligera** entre attendance y academic via API (sin transacciones distribuidas).

6. Impacto en pruebas

Las suites EP3 (JUnit/Mockito/JaCoCo + Vitest) siguen siendo la base. Tras las mejoras:

- Se mantienen tests de acceso/seguridad (AttendanceAccessServiceTest , etc.).
- El flujo de notify está diseñado para **no romper** el CRUD si la mensajería falla (adecuado para demo y para tests de servicio).
- Checklist de defensa incluye ejecutar mvn test en un servicio y npm test en frontend (ver 06).

Nota: los reportes HTML JaCoCo/Vitest de informe-ep3/jacoco-reports y coverage-frontend corresponden a la corrida EP3. Para el examen se recomienda regenerar al menos un reporte fresco en vivo o actualizar copias si el tiempo lo permite.

7. Conclusión

La evolución respecto a EP3 fortalece el **valor de negocio** del Libro Digital: no solo registra información, sino que **comunica de inmediato a la familia**. La mejora innovadora de alertas a apoderados, respaldada por mensajería TG, confirmaciones UX y arquitectura Database-per-Service, responde al feedback de la presentación N°3 y prepara una defensa ágil con software en ejecución, código, BD y pruebas.